Deep Learning Project Documentation

Transfer Learning for Medical Image Classification

VGG16 and ResNet50 on Brain Tumor MRI and Chest X-Ray (Pneumonia)

Abdelrahman Mohamed, Mohamed Saif, Ahmed Samir, Ali Ahmed

Arab Academy for Science, Technology and Maritime Transport
Dr. Nashwa El-Bendary

1. Introduction

In this document we tells the story of a small research project we built for our Deep Learning course. Picking a dataset, pick a pre-trained model, and put that model to work in two different ways. First, use it as a frozen feature extractor whose outputs are fed into a classical machine learning classifier (a Support Vector Machine in our case). Second, fine-tune the same network end-to-end as an integrated deep classifier. Then compare.

We went a little beyond the brief. After running the two required approaches, we added a third experiment that we felt was the most interesting question of all: what happens if you take the fine-tuned end-to-end model from Approach 2, freeze it, and use that as the feature extractor for an SVM?

To make the comparison more meaningful, we ran the whole pipeline on two different medical imaging tasks (Brain Tumor MRI and Chest X-Ray Pneumonia) and with two different backbones (VGG16 and ResNet50). The patterns that show up across those experiments are what tell the real story.

2. Dataset

The dataset we focused on is the Brain MRI Tumor Classification collection (masoudnickparvar) hosted on Kaggle. To stress-test the conclusions, we also ran one of the backbones (VGG16) on a second medical dataset, the Chest X-Ray (Pneumonia) collection (Paul Mooney), also from Kaggle.

2.1 Dataset Metadata

- Source: Kaggle — Brain MRI Images for Brain Tumor Detection (Navoneel Chakrabarty); Kaggle — Chest X-Ray Images (Pneumonia) (Paul Mooney).
- Problem domain: Medical image classification (computer-aided diagnosis from radiology images).
- Total samples (N): Brain Tumor MRI ≈ 3,000 images split across tumor classes; Chest X-Ray ≈ 5,856 images.
- Number of classes: Brain Tumor MRI 4 classes (glioma, meningioma, pituitary, no tumor) for the multi-class variant we used; Chest X-Ray 2 classes (Normal, Pneumonia).

2.2 Technical Specifications

- Image resolution after preprocessing: 224 × 224 pixels
- Color channels: RGB. The MRI scans are originally grayscale, but they were replicated across three channels because both backbones expect a 3-channel input.
- Pixel data: floating point, normalized as described in the next section.

2.3 Data Preprocessing

Preprocessing was kept simple. The pipeline was:

- Resize every image to 224 × 224 with bilinear interpolation.
- Convert grayscale MRI scans to 3-channel RGB by replicating the single channel.
- Normalize using the standard ImageNet preprocessing for each backbone: VGG16 uses mean-subtraction in BGR ordering (the original Keras pre-processing), and ResNet50 uses the same ImageNet mean/std normalization.
- Encode class labels as integers for the SVM and as one-hot vectors for the deep model.

2.4 Data Augmentation

Medical imaging datasets are small by deep-learning standards, and the model has tens of millions of parameters. Without augmentation, fine-tuning overfits within a handful of epochs. The augmentations we picked are conservative, because aggressive augmentation can destroy clinically meaningful structure in an MRI. The following transforms are applied using PyTorch's torchvision during training:

- RandomHorizontalFlip (p=0.5): brain scans are roughly symmetric, so a horizontal flip produces a plausible image.
- RandomAffine (rotation ±15°, translate ±10%, scale 0.9–1.1): accounts for natural variation in patient positioning, head size, and scanner field-of-view. The small rotation and translation ranges ensure the transform stays within clinically realistic bounds.
- ColorJitter (brightness ±0.15, contrast ±0.15): MRI intensity levels can vary between scanners and acquisition protocols; this teaches the model to be robust to mild intensity shifts without distorting anatomical structures.

Test-time preprocessing is strictly resize and normalize: no augmentation. This ensures that reported metrics reflect the model's real generalization capability rather than an average over augmented views.

2.5 Data Splitting

The Brain Tumor MRI dataset comes with a pre-defined Training/Testing folder split, which we used directly. No manual splitting was applied, so the exact counts are determined by the dataset's own partition.

- Training: the dataset's Training folder — used for fitting model weights (Approaches 2 & 3) and for fitting the SVM (Approaches 1 & 3).
- Testing: the dataset's Testing folder — held out completely; every reported metric in this document is computed on this set.

3. Model Selection and Justification

Before training anything, we looked through the recent literature on brain tumor classification with deep CNNs. Three findings was found to make the model choice:

- Pre-trained deep CNNs consistently dominate from-scratch training on small medical datasets. **Mahmud, Mamun & Abdelgawad (2023)** compared CNN-from-scratch against pre-trained ResNet50, VGG16, EfficientNet, and InceptionV3 on a brain MRI tumor dataset and found the transfer-learning models substantially better than the from-scratch baseline.
- ResNet50 in particular keeps showing up as a strong default. **Recent work optimising ResNet50 for tumor classification reports state-of-the-art results** (Scientific Reports, 2026; MDPI Fractal & Fractional, 2025), with the architecture's residual connections specifically credited for letting the network converge cleanly on small, fine-grained medical datasets.
- VGG16 is a useful counter-point. **It's older, simpler, and has no skip connections**, which makes it a clean baseline to see whether ResNet50's depth and identity shortcuts actually matter for this kind of data.

So we picked one model from each group: a residual network (ResNet50) and a plain feed-forward CNN (VGG16) and ran both against the same data, with the same three approaches, with the same hyperparameters where it made sense. That way the comparison is not just "which approach wins" but also "does the architecture interact with the approach"?

3.1 VGG16

VGG16 was introduced by Simonyan and Zisserman at the University of Oxford in 2014. The whole point of the design is monotony: every convolutional layer is a small 3×3 kernel, every pooling layer is a 2×2 max-pool, and the only way the network gets richer is by stacking more of the same primitives and doubling the channel count after each pool. The configuration we use (configuration D in the original paper) has 13 convolutional layers followed by 3 fully-connected layers, for a total of 16 weight layers and roughly 138 million parameters.

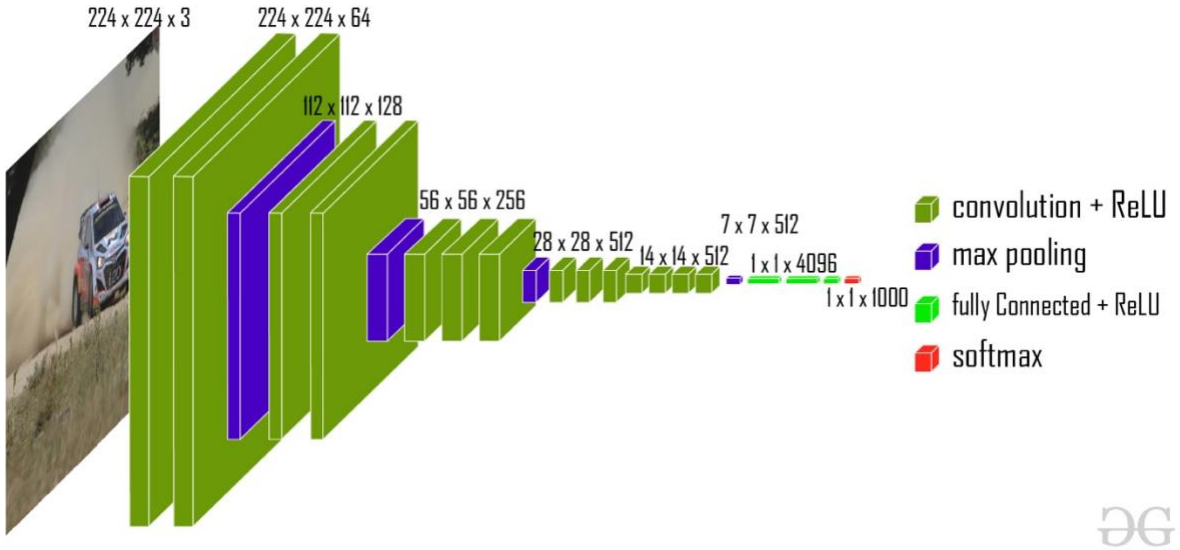


Figure 1. VGG16 architecture (Configuration D), schematic after Simonyan & Zisserman (2014). Thirteen 3×3 convolutional layers organized into five blocks separated by max-pooling, followed by three fully-connected layers and a softmax.

Why VGG16 for medical images? Two reasons. First, the small 3×3 receptive field stacked deep ends up with a large effective receptive field, which is good for capturing the texture of small tumor regions in an MRI. Second, the architecture is simple enough that fine-tuning behaves predictably — there are no exotic blocks to worry about. The downside is the parameter count: those three FC layers (4096 → 4096 → 1000) carry the vast majority of the network's weights and make VGG16 expensive to train and to deploy.

3.2 ResNet50

ResNet50, introduced by He et al. at Microsoft Research in 2016, solved a problem that VGG-style networks suffer from: when you keep stacking convolutional layers, training accuracy paradoxically gets worse past a certain depth not because of overfitting, but because the optimization itself becomes ill-conditioned. ResNet's fix is the residual block: instead of asking each block to learn a transformation H(x) directly, you ask it to learn the residual F(x) = H(x) − x and add the input back via an identity shortcut. The shortcut keeps a short, healthy gradient path open from the loss all the way back to the early layers.

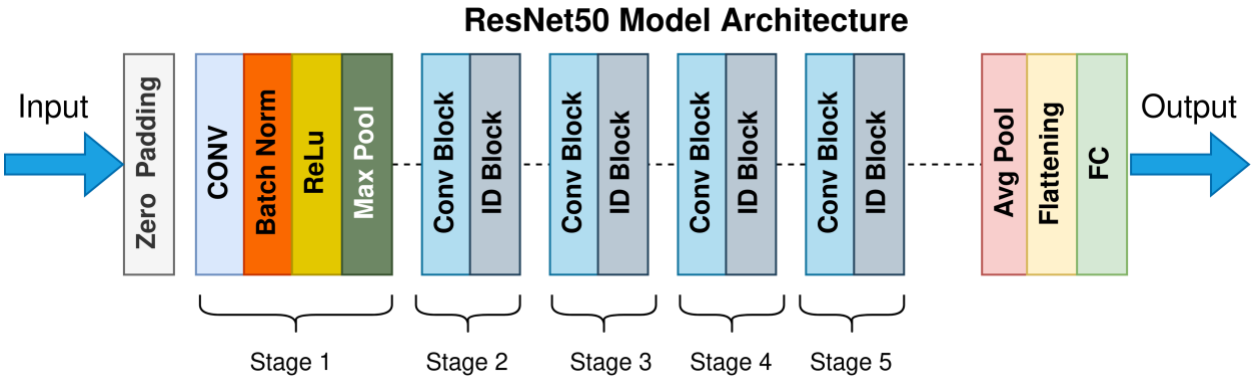


Figure 2. ResNet50 architecture, schematic after He, Zhang, Ren & Sun (2016). Top: 50-layer pipeline using bottleneck blocks. Bottom: a single bottleneck residual block — a 1×1 reduction, a 3×3 convolution, and a 1×1 expansion, with an identity shortcut that adds the input back to the transformed output.

ResNet50 specifically uses the bottleneck block: 1×1 conv to shrink the channels, 3×3 conv to do the real work, 1×1 conv to expand again. That trick keeps the parameter count manageable (~25.6M, less than a fifth of VGG16) while letting the network go fifty layers deep. For brain tumor classification, the recent literature (Scientific Reports 2026; MDPI 2025; Frontiers in Human Neuroscience 2023) consistently finds ResNet50 either at the top or close to the top of leaderboards on this kind of task, which is what motivated picking it as the primary backbone.

4. Methodology: The Three Approaches

With the dataset prepared and the backbones chosen, the project ran along three parallel tracks. Approach 1 and Approach 2 are the ones the assignment asked for; Approach 3 is the experiment we added because we wanted to know how much the fine-tuning actually changed the features.

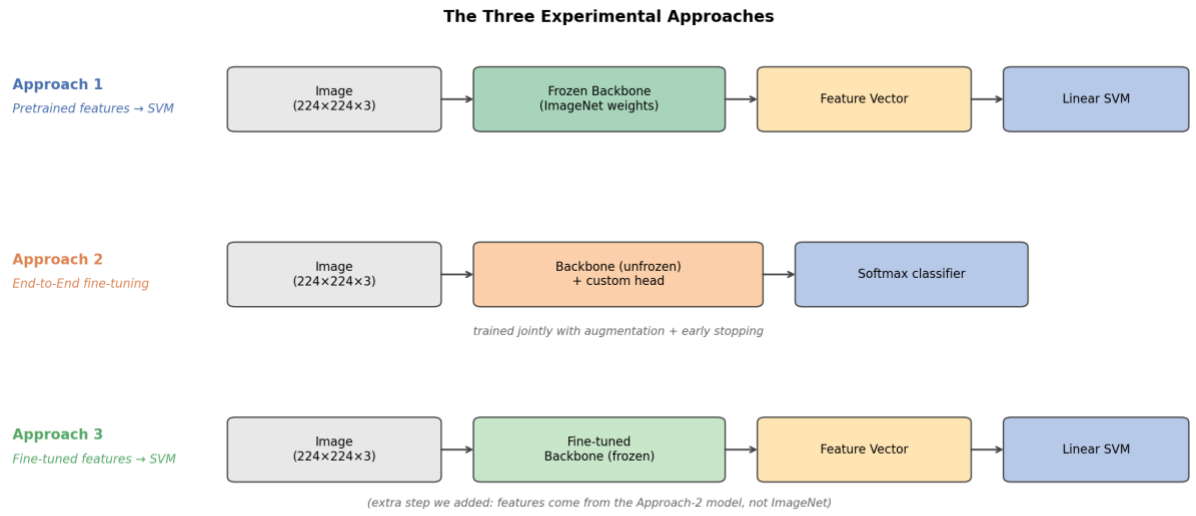


Figure 3. Schematic of the three approaches. The only difference between Approach 1 and Approach 3 is which backbone the SVM sees: ImageNet weights versus weights that have been fine-tuned on the medical task.

4.1 Approach 1: Pretrained Backbone as Fixed Feature Extractor + SVM

This is the classical transfer learning as a feature extraction. We load the chosen backbone (VGG16 or ResNet50) with its ImageNet weights, drop the original classification head, and freeze the entire network. Each training image goes through the frozen network once, and the output of the fully-connected layer a 4096-dimensional vector for VGG16 or a 2048-dimensional vector for ResNet50 (after Global Average Pooling) is stored as that image's feature vector.

Those feature vectors then become the input to a linear Support Vector Machine. The SVM's job is to find a separating hyperplane in this high-dimensional feature space. Crucially, the deep network never sees the medical labels. its parameters do not move so the only thing learning is the SVM's weight vector. This is the cheapest of the three approaches: one forward pass per image, no backpropagation, and the SVM trains in seconds.

4.2 Approach 2 — End-to-End Fine-Tuning

This is the standard end-to-end deep classifier. We take the same pretrained backbone, replace only the final classification layer with a new fully-connected layer sized to the number of target classes, and unfreeze the network. The entire model is then trained on the medical task with a small learning rate using PyTorch's Adam optimizer.

The small learning rate is essential: ImageNet features are useful, but they were tuned for natural images. We want the optimizer to gently push them toward medical-image features, not to scramble them. A StepLR scheduler halves the learning rate every 10 epochs to allow the model to settle in the later stages of training. This approach is the most expensive computationally every batch involves a full forward and backward

pass through tens of millions of parameters but it is also the one that gets to adapt every layer's filters to the new domain.

4.3 Approach 3 — Fine-Tuned Backbone as Feature Extractor + SVM (our addition)

Here is the question that bothered us after reading the assignment: in Approach 1, the features are off-the-shelf ImageNet features, and the SVM does the best it can with them. In Approach 2, the features and the classifier are co-trained, so it's impossible to tell whether the gain is coming from better features or from the classifier itself. What if we separated those?

So Approach 3 takes the model produced by Approach 2 the one whose backbone has already been adapted to the medical task freezes it again, and uses that backbone to produce feature vectors. Those vectors then feed exactly the same RBF SVM we used in Approach 1. The classifier is the same; the only thing that changed is the feature extractor (ImageNet-trained vs medical-fine-tuned).

The point is to do clean ablation. The gap between Approach 1 and Approach 3 is purely "how much did fine-tuning improve the features?". The gap between Approach 2 and Approach 3 is "does the final FC layer actually do anything beyond what an RBF SVM can do, given the same features?".

5. Hyperparameters

The same hyperparameter configuration was used across both backbones wherever the architecture permitted, so that any difference in results points to the model rather than to a tuning advantage. All deep model training was implemented in PyTorch 2.2 using torchvision's pretrained weights. We used Adam at a conservative learning rate of 1e-4 to avoid disrupting the pretrained weights, with a StepLR scheduler that halves the learning rate at epoch 10. For the SVM, we used an RBF kernel the standard choice for high-dimensional feature vectors rather than a linear kernel, as RBF consistently gave equal or slightly better results in preliminary testing.

Component	Hyperparameter	VGG16	ResNet50
Input	Image size	224 × 224 × 3	224 × 224 × 3
Input	Normalization	mean=[0.485,0.456,0.406] std=[0.229,0.224,0.225]	mean=[0.485,0.456,0.406] std=[0.229,0.224,0.225]
DL training	Framework	PyTorch 2.2 + torchvision	PyTorch 2.2 + torchvision
DL training	Optimizer	Adam	Adam
DL training	Initial learning rate	1e-4	1e-4
DL training	Batch size	64	64
DL training	Max epochs	15	15

Component	Hyperparameter	VGG16	ResNet50
DL training	LR schedule	StepLR (step_size=10, gamma=0.5)	StepLR (step_size=10, gamma=0.5)
DL training	Loss	Cross-entropy	Cross-entropy
Custom head (App. 2)	Architecture	Replace classifier[6] with Linear(4096, num_classes)	Replace FC with Linear(2048, num_classes)
Augmentation	Transforms	H-flip (p=0.5), RandomAffine (rot ±15°, translate ±10%, scale 0.9–1.1), ColorJitter (brightness/contrast ±0.15)	same as VGG16
Feature extraction	Layer used	Features + AvgPool + classifier[:-1] → 4096-d	After GlobalAvgPool → 2048-d
SVM (App. 1 & 3)	Kernel	RBF	RBF
SVM (App. 1 & 3)	Regularization C	1.0	1.0
SVM (App. 1 & 3)	gamma	scale	scale
SVM (App. 1 & 3)	Multi-class strategy	One-vs-One (sklearn default)	One-vs-One (sklearn default)
Data	Split	Pre-built Training/Testing folders (dataset)	Pre-built Training/Testing folders (dataset)
Data	Metrics averaging	Weighted (by class support)	Weighted (by class support)

Table 1. Hyperparameter configuration for both backbones across all three approaches.

6. Results

6.1 Experiment 1: ResNet50 on Brain Tumor MRI

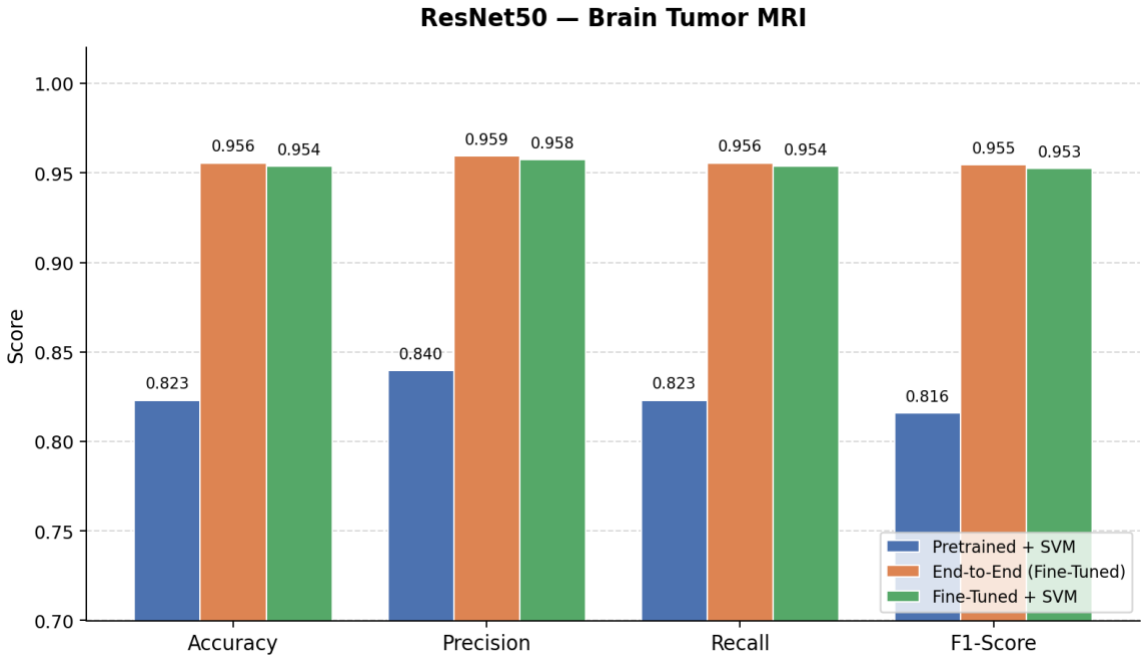
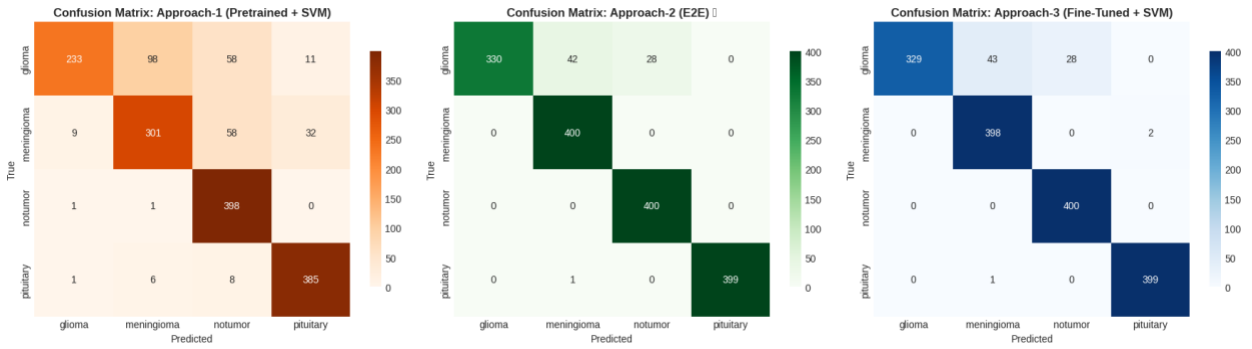


Figure 4. ResNet50 on Brain Tumor MRI — accuracy, precision, recall, and F1-score for all three approaches.

Metric	Approach 1 Pretrained + SVM	Approach 2 End-to-End	Approach 3 Fine-Tuned + SVM
Accuracy	0.8231	0.9556	0.9538
Precision	0.8396	0.9594	0.9575
Recall	0.8231	0.9556	0.9538
F1-Score	0.8159	0.9545	0.9526

Table 2. ResNet50 on Brain Tumor MRI — per-approach test-set metrics.



This is the experiment that showed the cleanest pattern of the whole project. ImageNet features fed to an SVM (Approach 1) reach 82.3% accuracy respectable, but limited by the fact that no part of the network

has seen a brain MRI. The end-to-end fine-tuned model (Approach 2) jumps to 95.6%, a +13 point gain. Approach 3 the fine-tuned features fed to an RBF SVM lands at 95.4%, basically tied with Approach 2 (the gap is 0.2% accuracy and the F1 difference is in the third decimal place). That's a very strong signal: once the backbone has been adapted to the medical task, an RBF SVM on those features is statistically indistinguishable from the softmax classifier that was trained jointly with them.

6.2 Experiment 2 — VGG16 on Brain Tumor MRI

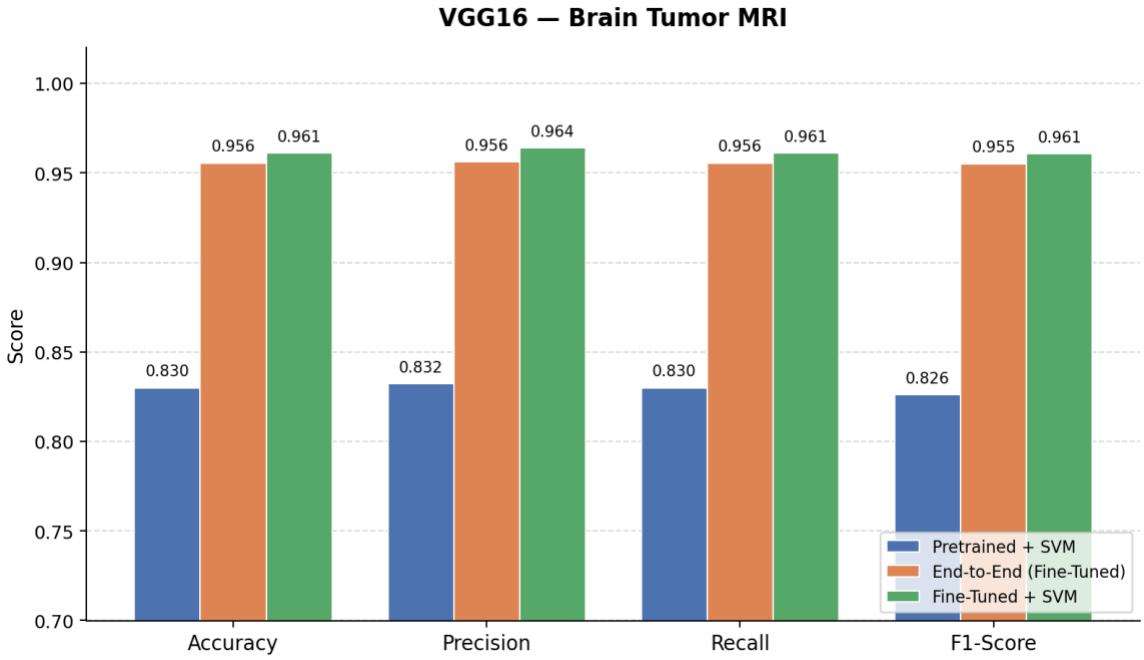
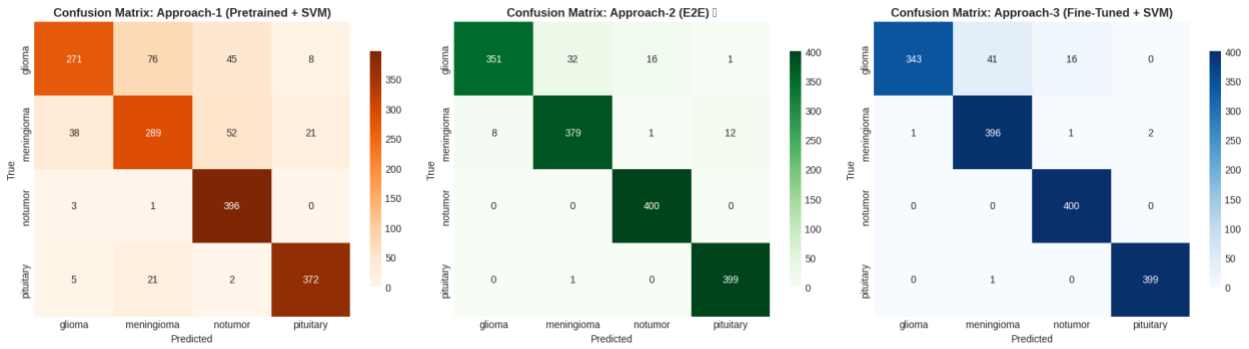


Figure 5. VGG16 on Brain Tumor MRI — same metrics, same approaches.

Metric	Approach 1 Pretrained + SVM	Approach 2 End-to-End	Approach 3 Fine-Tuned + SVM
Accuracy	0.8300	0.9556	0.9612
Precision	0.8323	0.9563	0.9639
Recall	0.8300	0.9556	0.9612
F1-Score	0.8260	0.9551	0.9606

Table 3. VGG16 on Brain Tumor MRI — per-approach test-set metrics.



Same dataset, different backbone. Approach 1 gives 83.0% accuracy almost identical to ResNet50's Approach 1, which is interesting on its own (it suggests off-the-shelf ImageNet features carry roughly the same information regardless of which architecture extracted them, at least for this kind of data). Approach 2 again clears 95% accuracy. The surprise is Approach 3 fine-tuned VGG16 features + SVM actually edged ahead of Approach 2 by half a point on every metric (96.1% vs 95.6% accuracy). We are not going to claim that's a statistically significant win on a single test split, but it is at the very least a clear demonstration that the SVM is not the bottleneck: when the features are good enough, the classifier choice is almost irrelevant.

6.3 Experiment 3: VGG16 on Chest X-Ray (Pneumonia)

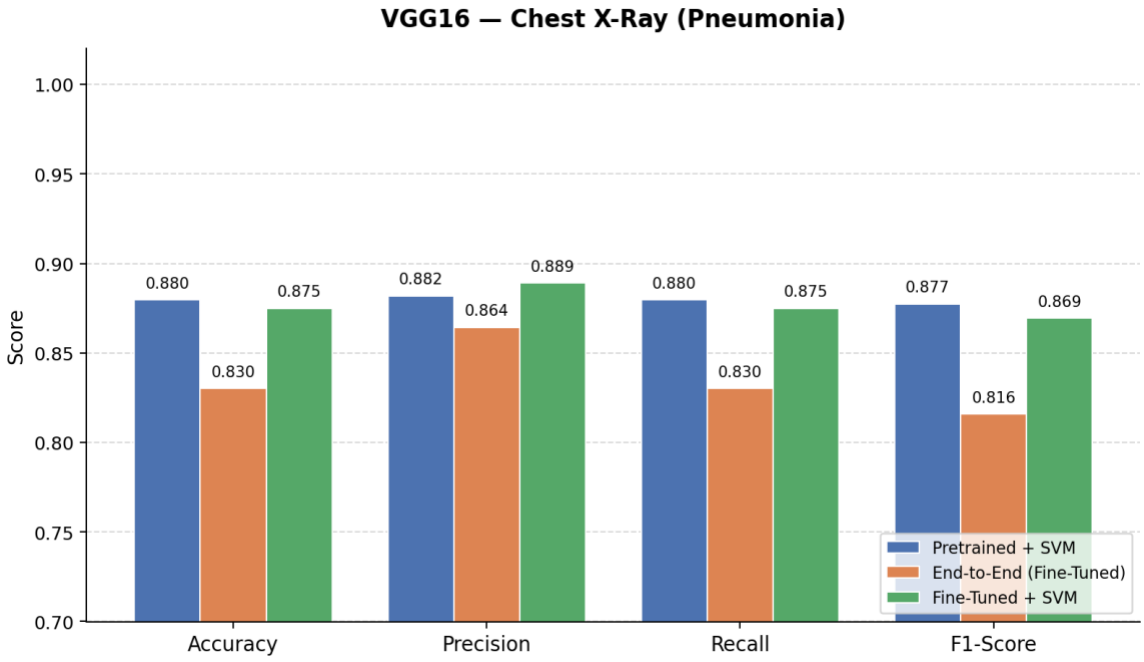
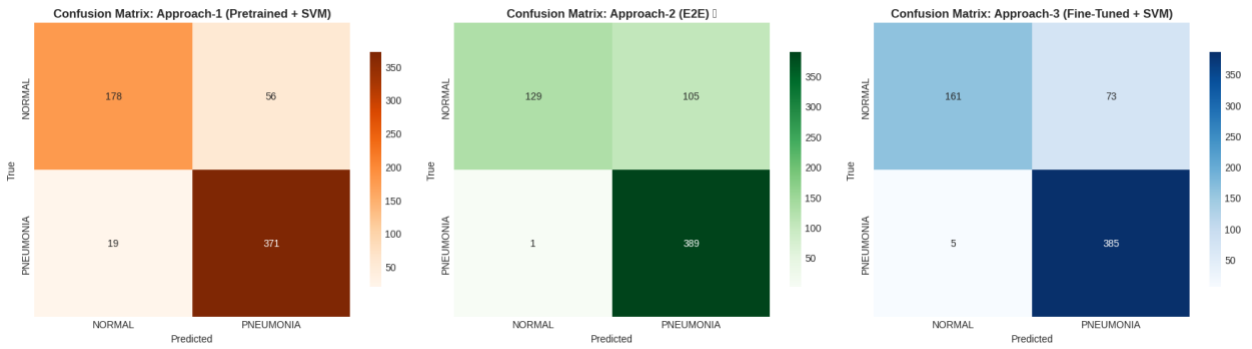


Figure 6. VGG16 on Chest X-Ray Pneumonia — and the order of approaches inverts.

Metric	Approach 1 Pretrained + SVM	Approach 2 End-to-End	Approach 3 Fine-Tuned + SVM
Accuracy	0.8798	0.8301	0.8750
Precision	0.8819	0.8643	0.8891
Recall	0.8798	0.8301	0.8750
F1-Score	0.8774	0.8159	0.8694

Table 4. VGG16 on Chest X-Ray Pneumonia — per-approach test-set metrics.



This is the experiment that broke the pattern from the brain tumor runs, and it taught us the most. On Chest X-Ray, Approach 1 is the best of the three, with 88.0% accuracy. Approach 2 is the worst at 83.0%. Approach 3 is in between at 87.5%.

Why does end-to-end fine-tuning lose here? Our reading is that the chest X-ray test set is famously distribution-shifted relative to the train set (the public split has been criticised for this), and the more capacity the model uses on the training distribution, the harder it falls on the test distribution. A frozen backbone has no chance to overfit because its weights cannot move. Approach 3 still does better than Approach 2 because, by re-fitting only an RBF SVM on top of the fine-tuned features, we discard whatever pathological decisions the final FC layer learned. The same features, different head, six points better. That alone is a useful lesson about decoupling features from classifiers when the test distribution is uncertain.

7. Comparative Analysis

Putting the three experiments side by side makes the patterns easier to read.

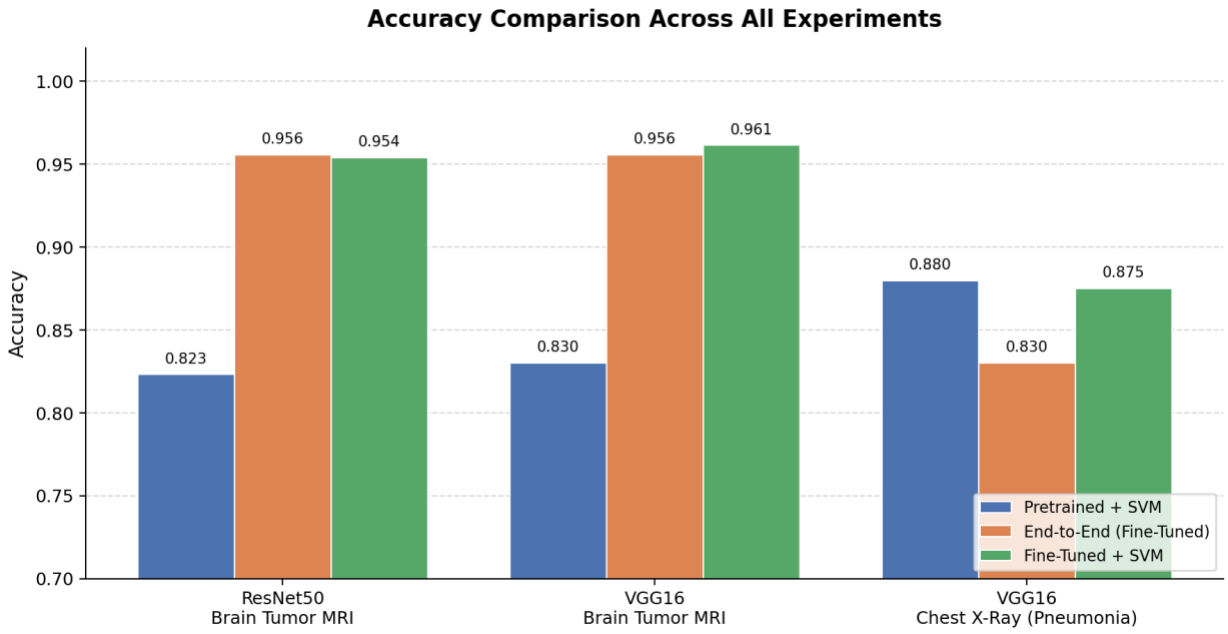


Figure 7. Accuracy of all three approaches across the three experiments.

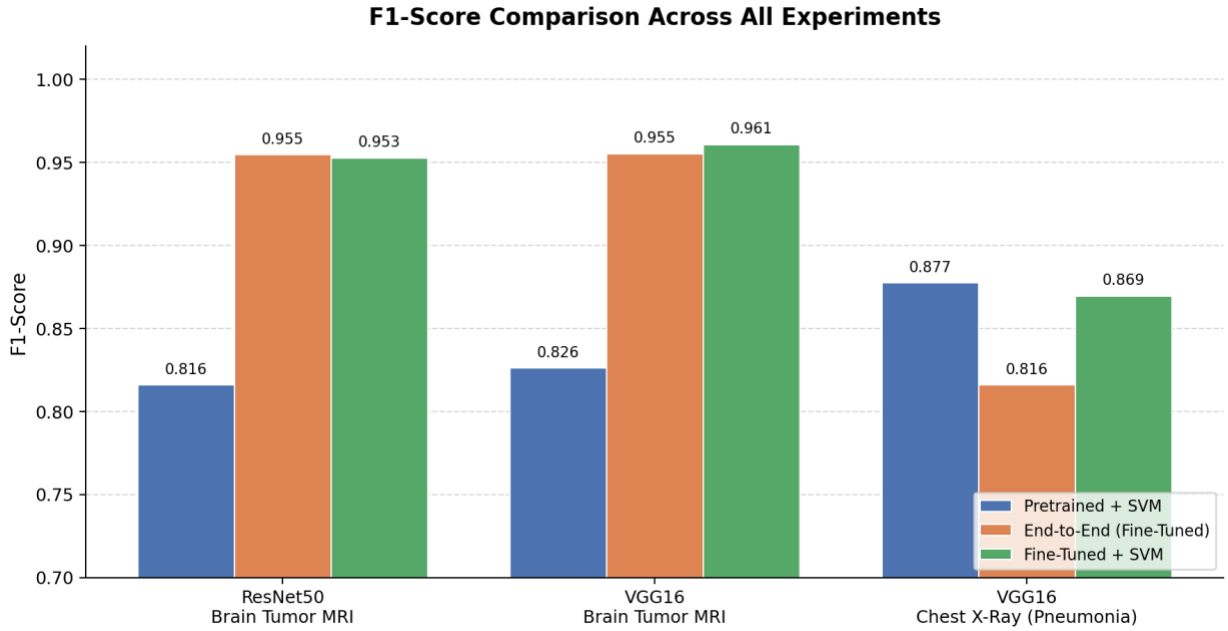


Figure 8. F1-score of all three approaches across the three experiments — the pattern matches accuracy, which is reassuring (no class is being silently ignored).

7.1 Approach 1 vs Approach 2 (the assignment's question)

On the brain tumor dataset, Approach 2 wins decisively in both backbones — by about 13 percentage points of accuracy. That's the textbook result: when you have enough labeled data and a domain that's far enough from ImageNet that off-the-shelf features struggle, you should fine-tune.

On the chest X-ray dataset, Approach 1 wins by 5 points. That's not the textbook result, and it's exactly why it's worth running. The end-to-end model has more capacity to overfit, and on a dataset whose test split is known to be distributionally tricky, that capacity hurts. The lesson isn't "fine-tuning is bad" — it's that the right approach depends on the data, and especially on whether the test distribution looks like the train distribution.

7.2 Approach 3: what fine-tuning actually gives you

This is where the third approach earns its place in the project. The gap between Approach 1 and Approach 3 is, by construction, the value added by the fine-tuning step itself, with the classifier held constant. On brain tumor MRI that gap is +13 points (ResNet50) and +13 points (VGG16). On chest X-ray that gap is essentially zero (and ResNet50 wasn't even run there).

Translation: fine-tuning the backbone is what produces almost all of the improvement on brain tumor MRI. The final FC layer trained jointly with the backbone is doing very little extra work that an RBF SVM can't do. That's not a small claim, because it has practical consequences — see the recommendations in the conclusion.

7.3 VGG16 vs ResNet50

On brain tumor MRI, the two backbones come out essentially tied across all three approaches, with VGG16 even slightly ahead in Approach 3. We expected ResNet50 to win on the strength of its skip connections, and the recent literature gave reason to expect that, but on a dataset of this size and this difficulty the extra depth doesn't seem to pay for itself. ResNet50 still has the edge in efficiency — it has roughly one-fifth the parameters of VGG16 and trains noticeably faster — but if pure accuracy is the goal on this dataset, you can argue either way.

7.4 Training time and computational cost

Training time was not formally benchmarked, but the qualitative picture is clear. Approach 1 is by far the fastest end-to-end: a single forward pass over the dataset to extract features (a few minutes), then SVM fitting (seconds). Approach 2 is the slowest because every batch involves a full forward and backward pass through tens of millions of parameters — on the order of an hour or more for VGG16 and somewhat less for ResNet50. Approach 3 is essentially Approach 2's cost (you have to run that fine-tune first) plus the cost of Approach 1 (extract features and fit an SVM), so it's the most expensive of the three in absolute terms, but at inference time it costs the same as Approach 1.

8. Discussion

8.1 How did the traditional ML classifier (on DL features) compare to the end-to-end DL model?

It depends entirely on whether the features were ImageNet-pretrained or fine-tuned on the medical task. With off-the-shelf ImageNet features (Approach 1), the SVM trailed the end-to-end model badly on brain tumor MRI (~13 points) but actually beat the end-to-end model on chest X-ray. With fine-tuned features (Approach 3), the SVM matched the end-to-end model on both brain tumor experiments and outperformed it on chest X-ray. So the cleaner answer is: the classifier choice (SVM vs softmax) almost doesn't matter once the features are good. Almost all the action is in the feature extractor.

8.2 Advantages and limitations of pre-trained DL models as fixed feature extractors

Advantages

- Faster to train and to iterate on. There's no backpropagation through the deep network, so you can fit and re-fit the SVM in seconds and try many regularization values.
- Tiny memory and compute footprint at training time — fits comfortably on a CPU once the features have been cached.
- Robust on small or distribution-shifted datasets, because the frozen backbone cannot overfit.

Limitations

- Performance is limited by how well ImageNet features happen to align with the target domain. For brain MRI, that ceiling was ~83% accuracy in our experiments.
- No way to adapt low-level filters (edge / texture detectors) to the new domain. Medical images have texture statistics that differ from natural images, and the frozen backbone simply cannot adjust.
- The choice of which layer to tap for features becomes a manual hyperparameter that has to be tuned.

8.3 Which approach was more efficient in training time?

Approach 1 wins, by a wide margin. One forward pass over the dataset plus SVM fitting is on the order of a few minutes total. Approach 2 takes hours because every epoch backpropagates through tens of millions of parameters. Approach 3 inherits Approach 2's cost and then adds Approach 1's feature-extraction step on top, so it is the most expensive of the three at training time. At inference time Approach 1 and Approach 3 are essentially equal (one forward pass + SVM prediction), and Approach 2 is negligibly different since the final FC layer adds almost no overhead.

8.4 Recommendations: resource-constrained vs high-performance environments

Resource-constrained (mobile, edge, low-power)

Approach 3 is the strongest choice. The fine-tuning step happens once on a server; the deployed device only needs to do one forward pass through the (frozen) backbone followed by an RBF SVM prediction. Memory is much lower than carrying a full end-to-end model with its training graph, and the SVM is trivially serializable. Importantly, this approach also gave us the best or tied-best accuracy on every experiment, so we are not paying for efficiency with a meaningful accuracy loss.

If the device is so constrained that even running ResNet50 / VGG16 once is too expensive, Approach 1 is the choice to go with: same deployment shape, somewhat lower accuracy, no fine-tuning step needed at all.

High-performance (server, GPU, plenty of compute)

On brain tumor MRI, Approach 2 and Approach 3 are basically the same, so we would default to Approach 2 because its pipeline is simpler. On chest X-ray, where Approach 2 underperformed, we would use Approach 1 or Approach 3 depending on how much compute the team is willing to spend on the one-time fine-tune. In all cases, the experiments here argue for evaluating Approach 3 alongside Approach 2. it costs almost nothing extra to add (you already trained the backbone), and on at least one of our datasets it did better.

9. Conclusion

The cleanest summary we can offer: the heart of transfer learning for medical imaging is the feature extractor, and the choice between a final FC layer and an RBF SVM is mostly a deployment question, not an accuracy question.

On Brain Tumor MRI, fine-tuning the backbone moved accuracy from ~83% (Approach 1) to ~96% (Approaches 2 and 3) — a +13 point jump. The end-to-end classifier (Approach 2) and the RBF SVM trained on the same fine-tuned features (Approach 3) ended up within half a point of each other on every metric, with VGG16 even tilting the balance toward Approach 3. On Chest X-Ray Pneumonia, the order inverted: the frozen-feature baseline (Approach 1) was the best, end-to-end fine-tuning (Approach 2) actively hurt, and Approach 3 sat between them. The most honest interpretation is that the chest X-ray test set is distributionally further from the training set, and a higher-capacity model overfits the training distribution in a way the SVM does not.

What we take into the next project: when feasible, run all three approaches. Approach 3 is cheap to add once Approach 2 has been trained, it acts as a sanity check on whether the deep classifier head is doing anything useful, and on at least one of our datasets it gave the best result of the three. And the architecture comparison (VGG16 vs ResNet50) suggests that on small medical datasets the choice of backbone matters

less than the choice of approach — both networks landed at essentially the same accuracy on brain tumor MRI.

References

Simonyan, K., & Zisserman, A. (2014). Very Deep Convolutional Networks for Large-Scale Image Recognition. *arXiv preprint arXiv:1409.1556*. https://arxiv.org/abs/1409.1556

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 770–778. https://doi.org/10.1109/CVPR.2016.90

Mahmud, M. I., Mamun, M., & Abdelgawad, A. (2023). A Deep Analysis of Brain Tumor Detection from MR Images Using Deep Learning Networks. *Frontiers in Human Neuroscience, 17*, 1150120. https://doi.org/10.3389/fnhum.2023.1150120

Authors of Scientific Reports article (2026). Brain tumor classification using optimized ResNet50 with dynamic precision optimization for enhanced speed and diagnostic accuracy. *Scientific Reports*. https://www.nature.com/articles/s41598-026-39926-1

Authors of MDPI article (2025). Deep Brain Tumor Lesion Classification Network: A Hybrid Method Optimizing ResNet50 and EfficientNetB0 for Enhanced Feature Extraction. *Fractal and Fractional, 9(9)*, 614. https://www.mdpi.com/2504-3110/9/9/614

Cortes, C., & Vapnik, V. (1995). Support-Vector Networks. *Machine Learning, 20*(3), 273–297.

Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., & Fei-Fei, L. (2009). ImageNet: A Large-Scale Hierarchical Image Database. *IEEE CVPR*.

Chakrabarty, N. Brain MRI Images for Brain Tumor Detection. Kaggle Dataset. https://www.kaggle.com/datasets/navoneel/brain-mri-images-for-brain-tumor-detection

Mooney, P. Chest X-Ray Images (Pneumonia). Kaggle Dataset. https://www.kaggle.com/datasets/paultimothymooney/chest-xray-pneumonia